



A simple one header solution to supporting utf8 strings in C and C++.

Functions provided from the C header `string.h` but with a `utf8*` prefix instead of the `str*` prefix:

API function docs

string.h	utf8.h	complete	C++14 constexpr
strcat	utf8cat	✓	
strchr	utf8chr	✓	✓
strcmp	utf8cmp	✓	✓
strcoll	utf8coll		
strcpy	utf8cpy	✓	
strcspn	utf8cspn	✓	✓
strdup	utf8dup	✓	
strfry	utf8fry		
strlen	utf8len	✓	✓
strnlen	utf8nlen	✓	✓
strncat	utf8ncat	✓	
strncmp	utf8ncmp	✓	✓
strncpy	utf8ncpy	✓	
strndup	utf8ndup	✓	

string.h	utf8.h	complete	C++14 constexpr
strpbrk	utf8pbrk	✓	✓
strrchr	utf8rchr	✓	✓
strsep	utf8sep		
strspn	utf8spn	✓	✓
strstr	utf8str	✓	✓
strtok	utf8tok		
strxfrm	utf8xfrm		

Functions provided from the C header strings.h but with a utf8* prefix instead of the str* prefix:

strings.h	utf8.h	complete	C++14 constexpr
strcasecmp	utf8casecmp	✓	✓
strncasecmp	utf8ncasecmp	✓	✓
strcasestr	utf8casestr	✓	✓

Functions provided that are unique to utf8.h:

utf8.h	complete	C++14 constexpr
utf8codepoint	✓	✓
utf8rcodepoint	✓	✓
utf8size	✓	✓
utf8size_lazy	✓	✓
utf8nsize_lazy	✓	✓
utf8valid	✓	✓
utf8nvalid	✓	✓
utf8makevalid	✓	

utf8.h	complete	C++14 constexpr
utf8codepointsize	✓	✓
utf8catcodepoint	✓	
utf8isupper	✗	✓
utf8islower	✗	✓
utf8lwr	✗	
utf8upr	✗	
utf8lwrcodepoint	✗	✓
utf8uprcodepoint	✗	✓

Usage

Just `#include "utf8.h"` in your code!

The current supported platforms are Linux, macOS and Windows.

The current supported compilers are gcc, clang, MSVC's `cl.exe`, and `clang-cl.exe`.

Design

The `utf8.h` API matches the `string.h` API as much as possible by design. There are a few major differences though.

`utf8.h` uses `char8_t*` in C++ 20 instead of `char*`

Anywhere in the `string.h` or `strings.h` documentation where it refers to 'bytes' I have changed that to utf8 codepoints. For instance, `utf8len` will return the number of utf8 codepoints in a utf8 string – which does not necessarily equate to the number of bytes.

API function docs

```
int utf8casecmp(const void *src1, const void *src2);
```

Return less than 0, 0, greater than 0 if `src1 < src2`, `src1 == src2`, `src1 > src2` respectively, case insensitive.

```
void *utf8cat(void *dst, const void *src);
```

Append the utf8 string `src` onto the utf8 string `dst`.

```
void *utf8chr(const void *src, utf8_int32_t chr);
```

Find the first match of the utf8 codepoint `chr` in the utf8 string `src`.

```
int utf8cmp(const void *src1, const void *src2);
```

Return less than 0, 0, greater than 0 if `src1 < src2`, `src1 == src2`, `src1 > src2` respectively.

```
void *utf8cpy(void *dst, const void *src);
```

Copy the utf8 string `src` onto the memory allocated in `dst`.

```
size_t utf8cspn(const void *src, const void *reject);
```

Number of utf8 codepoints in the utf8 string `src` that consists entirely of utf8 codepoints not from the utf8 string `reject`.

```
void *utf8dup(const void *src);
```

Duplicate the utf8 string `src` by getting its size, malloc'ing a new buffer copying over the data, and returning that. Or 0 if malloc failed.

```
size_t utf8len(const void *str);
```

Number of utf8 codepoints in the utf8 string `str` ,
excluding the null terminating byte.

```
size_t utf8nlen(const void *str, size_t n);
```

Similar to `utf8len` , except that only at most `n` bytes of `src` are looked.

```
int utf8ncasecmp(const void *src1, const void *src2, size_t n);
```

Return less than 0, 0, greater than 0 if `src1 < src2` , `src1 == src2` ,
`src1 > src2` respectively, case insensitive. Checking at most `n`
bytes of each utf8 string.

```
void *utf8ncat(void *dst, const void *src, size_t n);
```

Append the utf8 string `src` onto the utf8 string `dst` ,
writing at most `n+1` bytes. Can produce an invalid utf8
string if `n` falls partway through a utf8 codepoint.

```
int utf8ncmp(const void *src1, const void *src2, size_t n);
```

Return less than 0, 0, greater than 0 if `src1 < src2` ,
`src1 == src2` , `src1 > src2` respectively. Checking at most `n`
bytes of each utf8 string.

```
void *utf8ncpy(void *dst, const void *src, size_t n);
```

Copy the utf8 string `src` onto the memory allocated in `dst` .

Copies at most `n` bytes. If `n` falls partway through a utf8
codepoint, or if `dst` doesn't have enough room for a null

terminator, the final string will be cut short to preserve utf8 validity.

```
void *utf8pbrk(const void *str, const void *accept);
```

Locates the first occurrence in the utf8 string `str` of any byte in the utf8 string `accept`, or 0 if no match was found.

```
void *utf8rchr(const void *src, utf8_int32_t chr);
```

Find the last match of the utf8 codepoint `chr` in the utf8 string `src`.

```
size_t utf8size(const void *str);
```

Number of bytes in the utf8 string `str`, including the null terminating byte.

```
size_t utf8size_lazy(const void *str);
```

Similar to `utf8size`, except that the null terminating byte is excluded.

```
size_t utf8nsize_lazy(const void *str, size_t n);
```

Similar to `utf8size`, except that only at most `n` bytes of `src` are looked and the null terminating byte is excluded.

```
size_t utf8spn(const void *src, const void *accept);
```

Number of utf8 codepoints in the utf8 string `src` that consists entirely of utf8 codepoints from the utf8 string `accept`.

```
void *utf8str(const void *haystack, const void *needle);
```

The position of the utf8 string `needle` in the utf8 string `haystack`.

```
void *utf8casestr(const void *haystack, const void *needle);
```

The position of the utf8 string `needle` in the utf8 string `haystack`,
case insensitive.

```
void *utf8valid(const void *str);
```

Return 0 on success, or the position of the invalid utf8 codepoint on failure.

```
void *utf8nvalid(const void *str, size_t n);
```

Similar to `utf8valid`, except that only at most `n` bytes of `src` are looked.

```
int utf8makevalid(void *str, utf8_int32_t replacement);
```

Return 0 on success. Makes the `str` valid by replacing invalid sequences with
the 1-byte replacement codepoint.

```
void *utf8codepoint(const void *str, utf8_int32_t *out_codepoint);
```

Sets `out_codepoint` to the current utf8 codepoint in `str`, and returns the
address of the next utf8 codepoint after the current one in `str`.

```
void *utf8rcodepoint(const void *str, utf8_int32_t *out_codepoint);
```

Sets `out_codepoint` to the current utf8 codepoint in `str`, and returns the
address of the previous utf8 codepoint before the current one in `str`.

```
size_t utf8codepointsize(utf8_int32_t chr);
```

Returns the size of the given codepoint in bytes.

```
void *utf8catcodepoint(void *utf8_restrict str, utf8_int32_t chr)
```

Write a codepoint to the given string, and return the address to the next place after the written codepoint. Pass how many bytes left in the buffer to `n`. If there is not enough space for the codepoint, this function returns null.

```
int utf8islower(utf8_int32_t chr);
```

Returns 1 if the given character is lowercase, or 0 if it is not.

```
int utf8isupper(utf8_int32_t chr);
```

Returns 1 if the given character is uppercase, or 0 if it is not.

```
void utf8lwr(void *utf8_restrict str);
```

Transform the given string into all lowercase codepoints.

```
void utf8upr(void *utf8_restrict str);
```

Transform the given string into all uppercase codepoints.

```
utf8_int32_t utf8lwrcodepoint(utf8_int32_t cp);
```

Make a codepoint lower case if possible.

```
utf8_int32_t utf8uprcodepoint(utf8_int32_t cp);
```

Make a codepoint upper case if possible.

Codepoint Case

Various functions provided will do case insensitive compares, or transform utf8 strings from one case to another. Given the vastness of unicode, and the authors lack of understanding beyond latin codepoints on whether case means anything, the following categories are the only ones that will be checked in case insensitive code:

- [ASCII](#)
- [Latin-1 Supplement](#)
- [Latin Extended-A](#)
- [Latin Extended-B](#)
- [Greek and Coptic](#)
- [Cyrillic](#)

Todo

- Implement utf8coll (akin to strcoll).
- Implement utf8fry (akin to strfry).
- Investigate adding dst buffer sizes for utf8cpy and utf8cat to catch overwrites (as suggested by [@FlohOfWoe](#) in <https://twitter.com/FlohOfWoe/status/618669237771608064>)

License

This is free and unencumbered software released into the public domain.

Anyone is free to copy, modify, publish, use, compile, sell, or distribute this software, either in source code form or as a compiled binary, for any purpose, commercial or non-commercial, and by any means.

In jurisdictions that recognize copyright laws, the author or authors of this software dedicate any and all copyright interest in the software to the public domain. We make this dedication for the benefit of the public at large and to the detriment of our heirs and successors. We intend this dedication to be an overt act of relinquishment in perpetuity of all present and future rights to this software under copyright law.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

For more information, please refer to <http://unlicense.org/>